

NTREAT Architecture

NTREAT — Architecture (Backend + Frontend)

NTREAT is a microservices-based domain security scanning platform. A client submits a domain, the orchestrator fans that domain (and its discovered subdomains) out across 7 independent scanner services, each scanner writes its own findings straight into MongoDB, and a scoring layer turns those findings into a weighted 0-100 grade per domain and an overall scan grade. A React/TypeScript frontend consumes all of this through a single gateway and a single shared data-fetching context.

This document covers the whole stack: **Part I** is the backend (microservices, queues, MongoDB, scoring). **Part II** is the frontend (routing, data fetching, scan lifecycle from the UI's point of view).

Naming note: “orchestrator” is intentionally spelled `orchrestator` / `orchrestor` throughout folder names, container names, queue names, and URL paths (e.g. `orchrestor_service`, `ORCHRESTATOR_SERVICE_URL`, `/orchrestator/scan`). The one exception is the Python attribute `Config.ORCHESTRATOR_URL`, which stays correctly spelled. This is preserved intentionally — don't “fix” it.

Table of contents

Part I — Backend

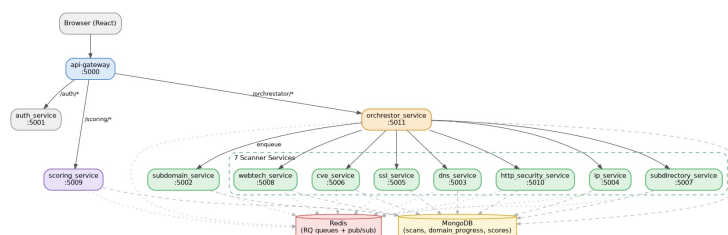
1. [Service topology](#)
2. [API gateway routing](#)
3. [High-level request flow](#)
4. [Scan kickoff — process_orchestration](#)
5. [Per-domain fan-out](#)
6. [How a scanner reports its result](#)
7. [Domain completion → scoring → finalization](#)
8. [Recovery path — the sweep job](#)
9. [Scoring system](#)
10. [Data model — how data is stored](#)
11. [API route reference](#)
12. [How data is retrieved — read paths](#)
13. [Worker pools](#)
14. [Key design decisions](#)

Part II — Frontend

15. [Frontend ↔ backend linkup](#)
 16. [Frontend routing](#)
 17. [Centralized data fetching — ScanContext](#)
 18. [Starting a scan from the UI](#)
 19. [Overview page logic](#)
-

Part I — Backend

1. Service topology

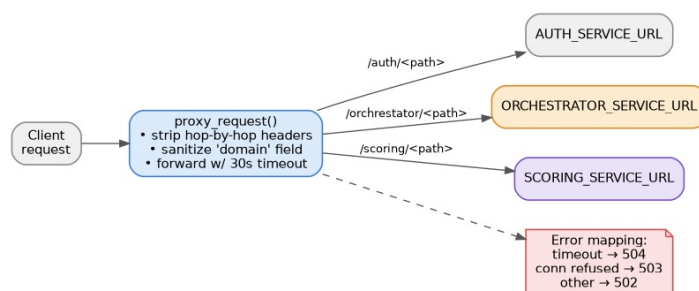


Service topology

Service	Port	Responsibility
api-gateway	5000	Single public entrypoint. Proxies /auth, /orchrestator, /scoring to internal services. Sanitizes the domain field on every proxied request body.
auth_service	5001	Authentication (JWT).
subdomain_service	5002	Discovers subdomains for the root domain. Runs once per scan, not per domain.
dns_service	5003	DNS security posture (SPF, DMARC, DKIM, DNSSEC, CAA, MX).
ip_service	5004	Open port exposure, reverse DNS, geolocation anomalies.
ssl_service	5005	Certificate validity, cipher strength, TLS version, HSTS.
cve_service	5006	Version fingerprinting + known-CVE matching.
subdirectory_service	5007	Brute-forces common paths; flags sensitive/admin/login paths.
webtech_service	5008	Technology fingerprinting via the Wappalyzer JSON database (7,193 signatures).
scoring_service	5009	Computes weighted scores, serves scan summaries/streams to the frontend.
http_security_service	5010	Security headers, cookies, CORS, dangerous files, login-wall classification.
orchrestor_service	5011	Owns a scan end-to-end: kicks off subdomain discovery, fans out scan jobs, tracks progress, triggers scoring,

fast_worker / slow_worker	—	finalizes the scan. Two RQ worker pools consuming different queues, so slow scanners never starve fast ones.
shared/	—	config.py, database.py, redis.py, orchestrator_client.py — common code every service imports so behavior can't silently drift between services.

2. API gateway routing



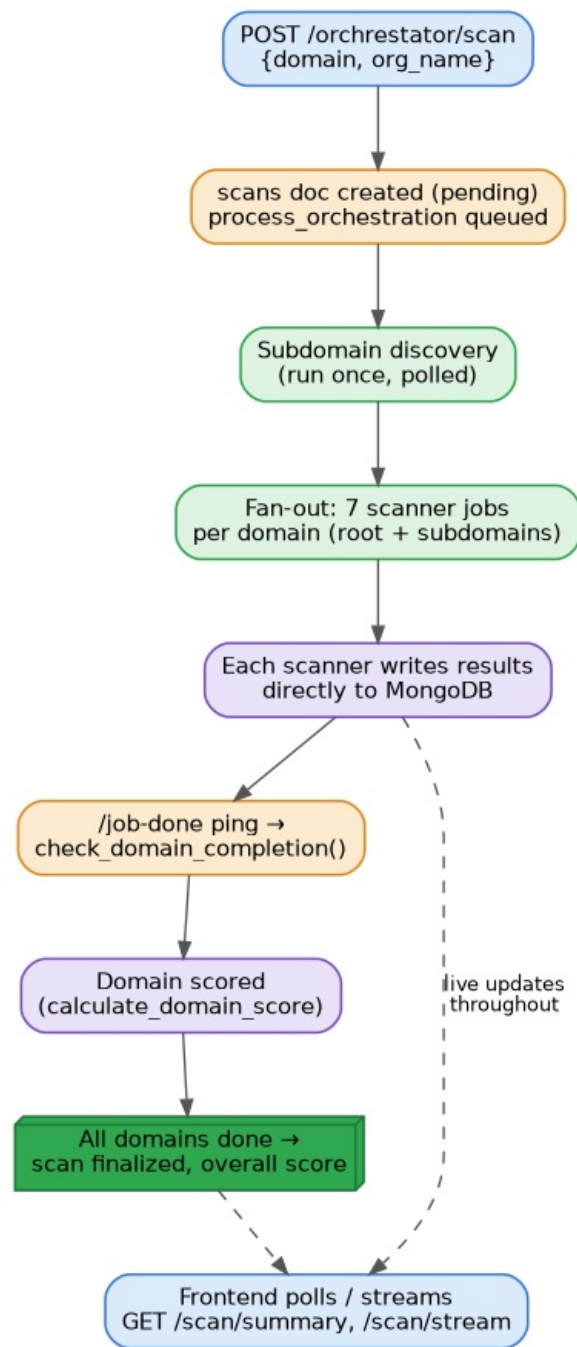
API gateway routing

The gateway is a thin reverse proxy (`proxy_request()` in `api-gateway/app.py`). For every request it:

1. Strips hop-by-hop headers (host, content-length, connection, transfer-encoding) before forwarding.
2. If the body is JSON and contains a domain field, strips a leading `http://https://` and any trailing slash — so every downstream service always receives a bare domain string, regardless of what the client typed.
3. Forwards method, headers, body, and cookies to the target service with a 30s timeout, and relays the response back — filtering out headers that would conflict with Flask's own response handling (content-encoding, content-length, transfer-encoding, connection).
4. Maps connection failures to clean HTTP errors: 504 on timeout, 503 on connection refused, 502 on any other request exception.

Gateway path	Forwards to
/auth/<path>	AUTH_SERVICE_URL
/orchrestator/<path>	ORCHRESTATOR_SERVICE_URL
/scoring/<path>	SCORING_SERVICE_URL

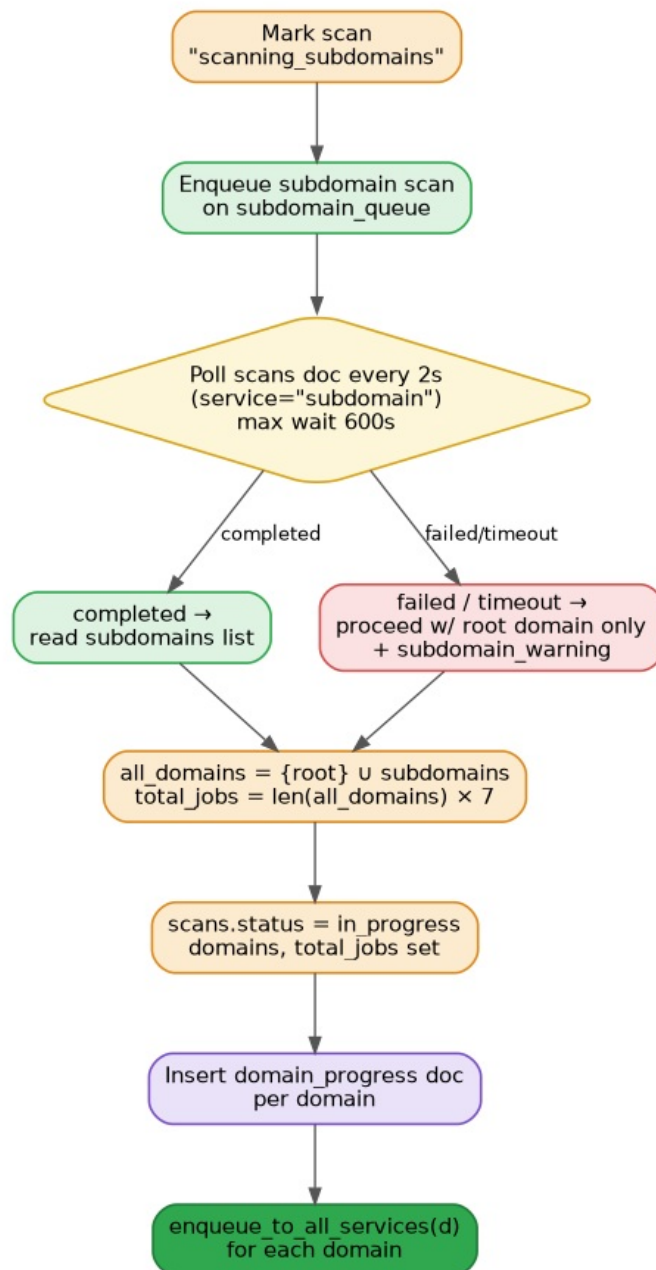
3. High-level request flow



High-level request flow

A scan starts with POST /orchrestator/scan and ends with every domain (root + subdomains) scored and the overall scan finalized. Everything in between — subdomain discovery, fan-out, per-scanner writes, completion detection, scoring — is asynchronous and driven by Redis-queued jobs, not by the original HTTP request staying open.

4. Scan kickoff — process_orchestration



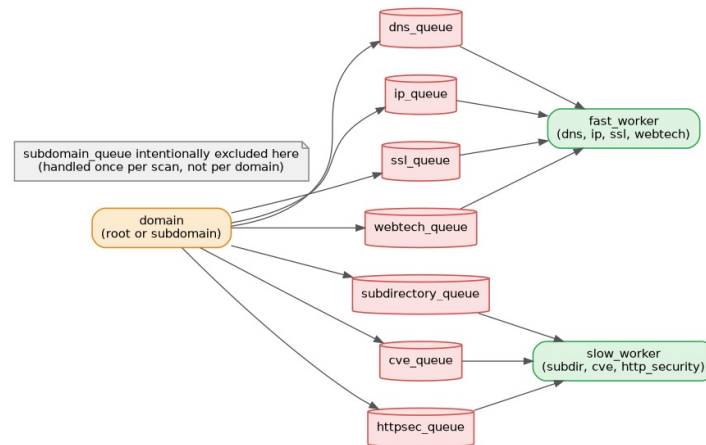
process_orchestration flow

This is the orchestrator's first background job, run once per scan (orchrestor_service/tasks.py):

1. Marks the scan scanning_subdomains in the scans collection.
2. Enqueues subdomain discovery and **polls** the scans document every 2 seconds (up to 600s) for service=subdomain to reach completed or failed.
3. On success, reads the discovered subdomain list. On failure or timeout, proceeds with just the root domain and records a subdomain_warning.
4. Computes all_domains = {root} u subdomains and total_jobs = len(all_domains) x 7.
5. Inserts one domain_progress document per domain, then calls enqueue_to_all_services() for each domain to fan out its 7 scanner jobs.

Why poll instead of fanning out immediately? Every downstream scanner needs the final subdomain list — the fan-out can't know how many domains it's scanning until subdomain discovery finishes (or times out).

5. Per-domain fan-out

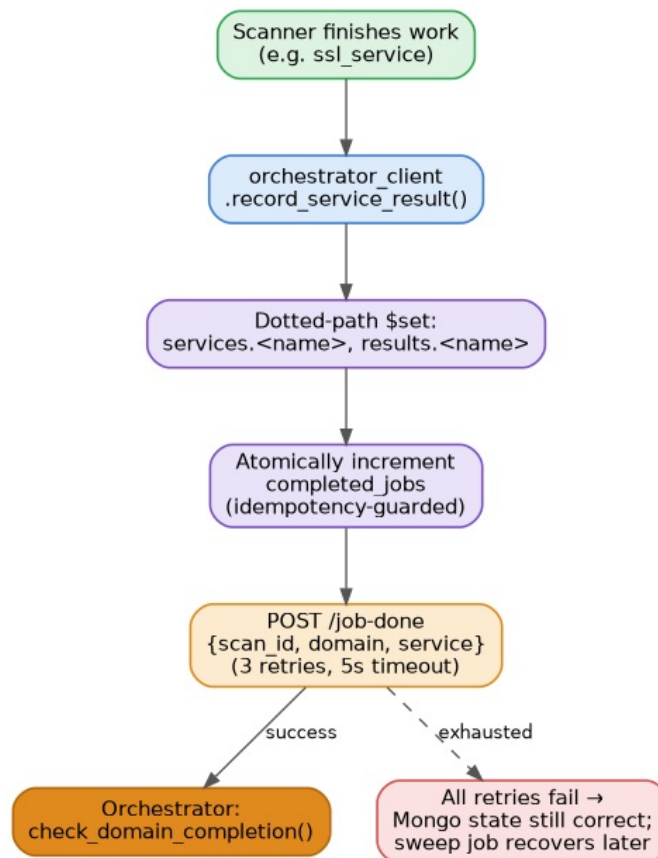


Per-domain fan-out

Each domain (root or subdomain) gets exactly 7 jobs — one per scanner — enqueued onto separate named queues (`queue_client.py`'s `SERVICE_TASK_MAP`).

`subdomain_queue` is deliberately **not** in `SERVICE_TASK_MAP` — it's handled once per scan in step 4 above, not once per domain. (This mirrors a real bug that was fixed: `subdomain_queue` had accidentally been included here before, which caused recursive subdomain scanning on every domain.)

6. How a scanner reports its result

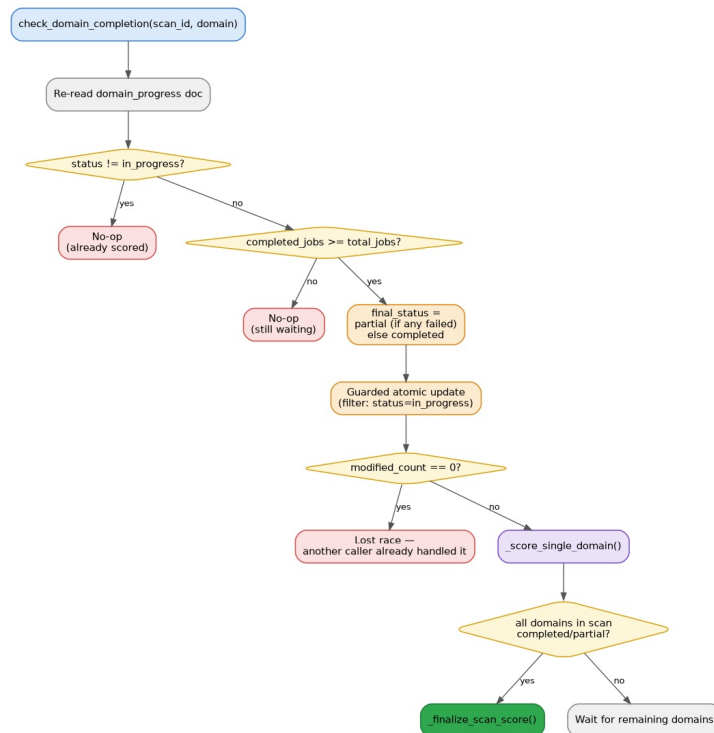


record_service_result flow

Every scanner service — regardless of which one — funnels through the same shared function, `orchestrator_client.record_service_result()`, so the write pattern can never drift between services:

- **No cross-scanner overwrites.** Every write uses a dotted MongoDB path (`services.<service>, results.<service>`), so concurrent scanners for the same domain can never clobber each other's data. This replaced an earlier version where all scanners wrote to one shared scans document.
- **completed_jobs is incremented atomically in the same write**, guarded by an idempotency check — a retried call for a service that already reported a terminal status (completed/failed) will not double-count.
- **Results never travel over HTTP.** Only a small, fixed-size completion ping (`POST /job-done`) goes to the orchestrator, with retries (3 attempts, 5s timeout each). If all retries fail, the data is still correct in Mongo — the sweep job (section 8) catches it later.

7. Domain completion → scoring → finalization

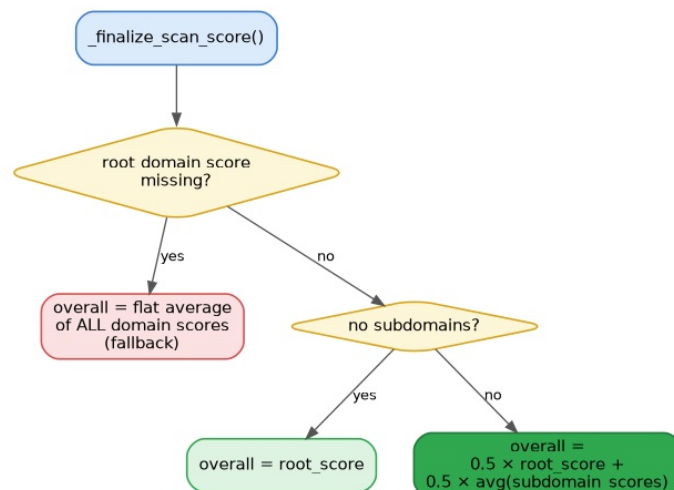


Domain completion, scoring, finalization

`check_domain_completion()` is shared by both `/job-done` (the normal path) and the sweep job (the recovery path), so completion logic lives in exactly one place:

1. Re-reads the domain's `domain_progress` doc.
2. If it's not `in_progress` anymore, it's already been scored — no-op.
3. If `completed_jobs < total_jobs`, still waiting on more scanners — no-op.
4. Otherwise, computes `final_status` (completed, or partial if any service failed) and does a **guarded** atomic update (`{"status": "in_progress"}` in the filter) so a concurrent caller (a retried `/job-done` ping racing the sweep job) can't double-score it.
5. Whoever wins the race calls `_score_single_domain()`, then checks whether every domain in the scan is now completed/partial. If so, calls `_finalize_scan_score()`.

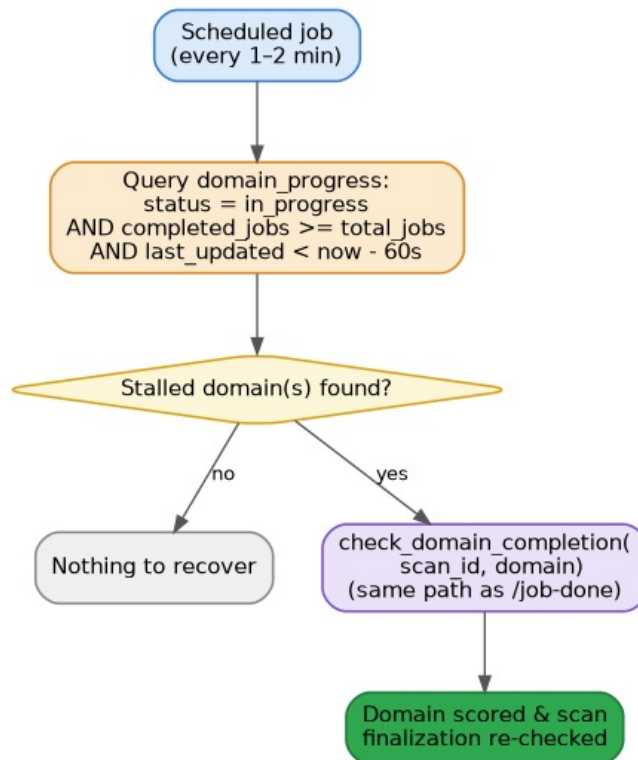
Overall scan score formula



Overall scan score formula

```
if root domain score is missing:
    overall = flat average of all domain scores    # fallback
elif no subdomains:
    overall = root domain score
else:
    overall = 0.5 × root_score + 0.5 × average(subdomain_scores)
```

8. Recovery path — the sweep job



Sweep recovery job

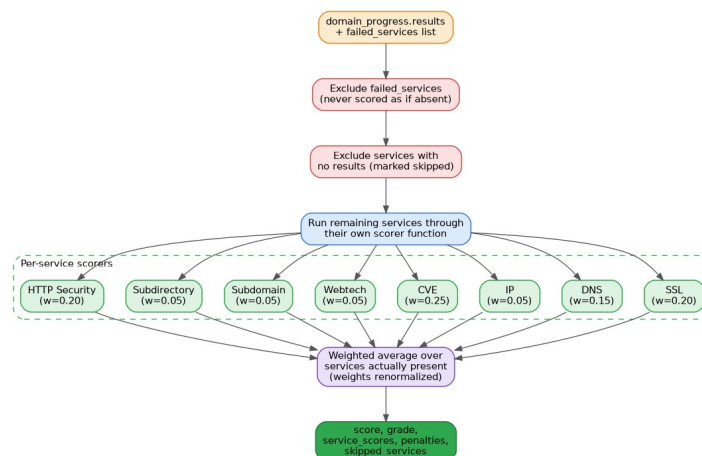
If `notify_orchestrator()` exhausts its 3 retries (the orchestrator was briefly unreachable), the domain's data is already correct in Mongo — nothing ever told the orchestrator to look, though.

`sweep_stalled_domains()` (intended to run on a schedule, every 1-2 minutes) finds any `domain_progress` doc that:

- is still `in_progress`,
- has `completed_jobs >= total_jobs` (i.e. actually finished by its own counters), and
- hasn't been touched in over 60 seconds (`STALE_AFTER_SECONDS`) —

and re-runs it through `check_domain_completion()`, the exact same path a normal `/job-done` call uses. The 60-second staleness window exists so the sweep never races a domain that's still actively mid-scan and about to get its own `/job-done` ping any second.

9. Scoring system



Scoring system flow

scoring_service/logic.py's `calculate_domain_score(results, failed_services)`:

1. **Excludes failed services entirely** — a service that errored out is never scored as if it were simply missing (which would previously default to a perfect 100).
2. **Excludes services with no results at all**, marking them skipped.
3. Runs each remaining service's raw output through its own scorer function.
4. Computes a **weighted average over only the services actually present** — weights are renormalized, so a missing/failed service's weight is excluded from the denominator rather than silently zeroing part of the total.

Service weights

Service	Weight
CVE	0.25
SSL	0.20
HTTP security	0.20
DNS	0.15
IP	0.05
Webtech	0.05
Subdomain	0.05
Subdirectory	0.05

Per-scanner scoring logic

- **SSL** — starts at 100. Penalizes an unreachable certificate (-40), failed verification e.g. expired/self-signed/mismatch (-25), weak ciphers (up to -20), legacy TLS 1.0/1.1 (-15), missing TLS 1.3 (-10), missing HSTS (-15), and an expiring/expired certificate (-15 to -30 depending on days remaining).
- **DNS** — starts at 100. Penalizes missing SPF (-20), missing DMARC (-20), no DKIM selector (-10), no DNSSEC (-15), no CAA (-10), no MX (-10).
- **IP** — starts at 100. Penalizes risky open ports — FTP, Telnet, RDP, VNC, MSSQL, MySQL, MongoDB — at -10 each (cap -40), missing reverse DNS (-10), and flagged geolocation anomalies (-10). No IP reputation scoring by design.
- **CVE** — starts at 100. Deducts by CVSS severity: critical $\geq 9.0 \rightarrow -20$ each (cap -60), high 7.0-8.9 $\rightarrow -10$ each (cap -40), medium 4.0-6.9 $\rightarrow -5$ each (cap -20), low $< 4.0 \rightarrow -2$ each (cap -5). If version

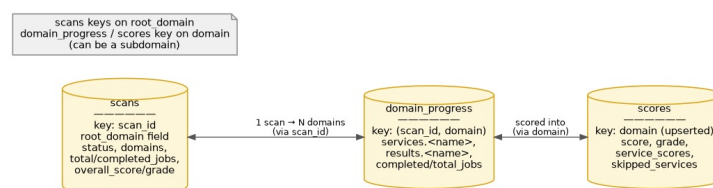
fingerprinting wasn't possible, the scan is *skipped*, not penalized.

- **Subdirectory** — starts at 100. Penalizes sensitive exposed paths like `.git/.env/backups/config` (up to -40), admin panels (-20), open directory listings (-15), login pages (-5).
- **HTTP security** — trusts the score/grade `http_security_service` already computed for itself (cookies, CORS, login-wall classification, dangerous files) rather than re-deriving it from flat headers. Falls back to a flat 0/F only for legacy scan docs missing that field.

Grade scale (_grade)

Score	Grade	Score	Grade
≥ 95	A+	≥ 65	C+
≥ 90	A	≥ 60	C
≥ 85	A-	≥ 55	C-
≥ 80	B+	< 55	F
≥ 75	B		
≥ 70	B-		

10. Data model — how data is stored



Data model across collections

NTREAT uses three MongoDB collections. **Key distinction:** scans documents key on `root_domain`, while `domain_progress` and `scores` key on `domain` (which can be a subdomain). Several past bugs traced back to querying scans with the wrong field name.

scans — one document per scan_id

Created by `POST /scan`, updated throughout the scan's lifecycle by the orchestrator.

Field	Written by	Meaning
<code>scan_id</code>	<code>/scan</code>	UUID for the whole scan
<code>root_domain, org_name</code>	<code>/scan</code>	What was requested
<code>status</code>	orchestrator	pending → scanning_subdomains → in_progress → completed/failed
<code>domains</code>	process_orchestration	Root + all discovered subdomains
<code>total_jobs, completed_jobs</code>	process_orchestration	$\text{len}(\text{domains}) \times 7$
<code>job_ids</code>	process_orchestration	{domain: {queue_name: rq_job_id}}
<code>overall_score, overall_grade</code>	<code>_finalize_scan_score</code>	Final weighted result

root_domain_score, domain_scores	_finalize_scan_score	Per-domain breakdown
domain_name	POST /domain-name	Optional friendly display label

domain_progress — one document per (scan_id, domain) pair

Inserted by process_orchestration when a domain is queued for scanning; updated by every scanner via record_service_result().

Field	Written by	Meaning
scan_id, domain	orchestrator (insert)	Identity
status	orchestrator	in_progress → completed / partial
total_jobs, completed_jobs	orchestrator / scanners	7 fixed; incremented atomically per scanner report
services.<name>	each scanner	{status: pending/completed/failed, error}
results.<name>	each scanner	Raw scanner output, only set on completed
last_updated	each scanner	Used by the sweep job's staleness check

Each scanner writes **only** its own services.<name> / results.<name> sub-path via a dotted \$set — this is what makes concurrent writes for the same domain safe.

scores — one document per domain (latest only, upserted)

Written by _score_single_domain (orchestrator, after each domain completes) and by POST /calculate (on-demand recompute from scoring_service).

Field	Meaning
domain, scan_id	Identity, plus which scan produced this score
score, grade	Overall weighted result for this domain
service_scores	Per-service score/grade/details/penalties breakdown
components_analyzed, skipped_services	Which services counted, which didn't and why
service_weights	The weight table used for this calculation (traceability)
calculated_at	Timestamp

11. API route reference

Gateway (api-gateway, port 5000)

Method	Path	Description
--------	------	-------------

GET	/	Service identity check
GET	/health	Health check
ANY	/auth/<path>	Proxied to auth_service
ANY	/orchrestator/<path>	Proxied to orchrestor_service
ANY	/scoring/<path>	Proxied to scoring_service

Orchestrator (orchrestor_service, port 5011 — reached via /orchrestator/*)

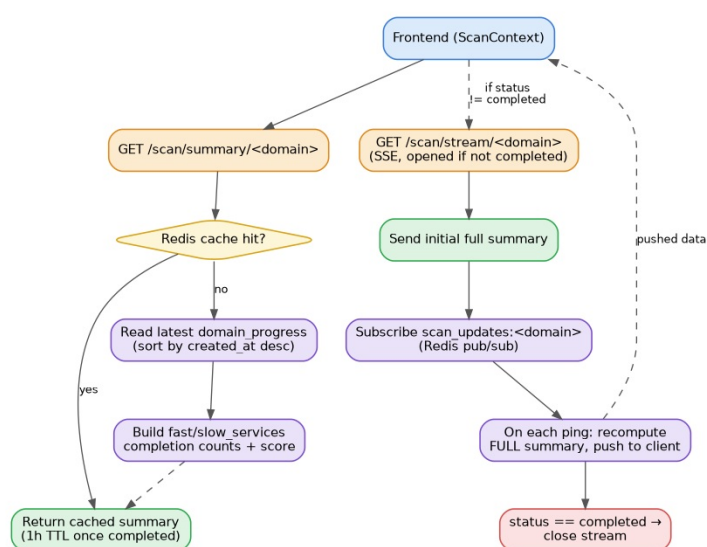
Method	Path	Description	DB effect
GET	/health	Health check	none
POST	/scan	Starts a new scan for a domain. Body: {domain, org_name}	Inserts into scan enqueues process_orchest
GET	/scan/<scan_id>	Returns the current scans document	Read-only
POST	/job-done	Called by scanners after record_service_result(). Body: {scan_id, domain, service}	Triggers check_domain_co — may update domain_progress and scans

Scoring service (scoring_service, port 5009 — reached via /scoring/*)

Method	Path	Description	DB effect
GET	/health	Health check	none
POST	/domain-name	Sets a friendly label for a root domain's scan(s). Body: {domain, domain_name}	update_many on scans (matches root_domain)
POST	/calculate	On-demand rescoring of a domain from its latest domain_progress doc. Body: {domain}	Upserts into scores
GET	/domain/<domain>	Returns the stored score document for a domain	Read-only (scores)
GET	/scans	Lists scans, optional ? domain=, ? include_results=, ?limit=	Read-only (scans)
GET	/scans/recent	Deprecated — last 100 scans, minimal fields	Read-only (scans)
GET	/scan/summary/<domain>	Consolidated status + per-service results + score for a domain	Read-only, Redis-cached 1h once completed
		Server-Sent Events stream of	Read-only,

GET	/scan/stream/<domain>	live summary updates	driven by Redis pub/sub
DELETE	/scans/<domain>	Deletes all scan data for a root domain	Deletes from scans and domain_progress clears Redis cache
GET	/scans/grouped	One row per scan_id for a dashboard/list view	Read-only (scans)

12. How data is retrieved — read paths



Frontend-facing read paths

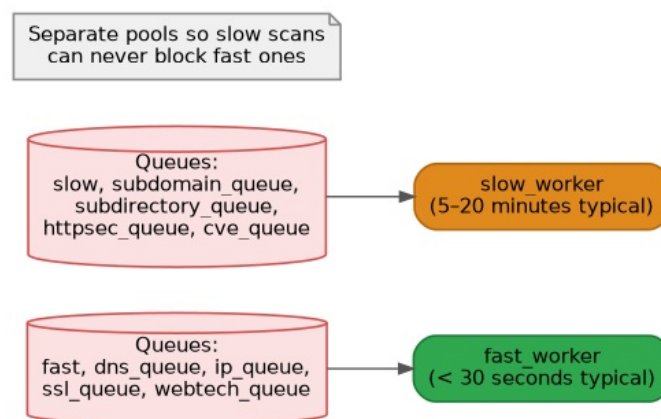
GET /scan/summary/<domain> — the main endpoint the frontend polls:

1. Checks Redis for a cached summary (scan_summary:<domain>); returns it immediately if present.
2. Otherwise reads the *latest* domain_progress document for that domain (across any scan_id — sort=[("created_at", -1)]).
3. Builds a per-service status map, split into fast_services (dns, ip, ssl, webtech) and slow_services (subdirectory, cve, http_security) completion counts.
4. If the domain's overall status is completed, fetches (or computes, as a fallback) its scores document and includes it in the response.
5. Caches the completed response in Redis for 1 hour.

GET /scan/stream/<domain> — SSE for live updates:

1. Sends one initial full summary (same shape as /scan/summary) immediately.
2. Subscribes to the scan_updates:<domain> Redis pub/sub channel.
3. On every pubsub message, **recomputes the full summary from scratch** (never relays the raw per-service pubsub payload) and pushes it — the frontend's handler expects data.scans on every message, and only closes the connection once overall status == "completed".

13. Worker pools



Worker pools

Two RQ worker processes consume different queues so slow scans can never block fast ones:

Worker	Queues	Typical duration
fast_worker	fast, dns_queue, ip_queue, ssl_queue, webtech_queue	< 30 seconds
slow_worker	slow, subdomain_queue, subdirectory_queue, httpsec_queue, cve_queue	5-20 minutes

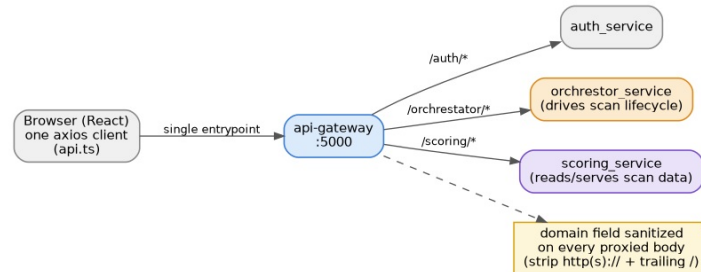
14. Key design decisions

- **Direct-to-Mongo writes over HTTP callbacks** — scanner results go straight into `domain_progress`; only a small completion signal travels over HTTP to the orchestrator, keeping `/job-done` fixed-size regardless of an individual scanner's payload.
- **Dotted-path \$set per service** — prevents concurrent scanners for the same domain from overwriting each other's results.
- **completed_jobs incremented atomically at write time**, not in a separate HTTP hop, so the counter can never drift out of sync with the data that's supposed to prove the job finished.
- **Idempotency guard on terminal status** — a retried scanner call can't double-increment `completed_jobs`.
- **Sweep job as a safety net, not the primary path** — recovers any domain whose `/job-done` ping never made it through, using the exact same completion logic as the normal path.
- **Weight renormalization in scoring** — failed/missing services are excluded from the denominator, never silently scored as 100.
- **scans keyed on root_domain, domain_progress/scores keyed on domain** — deliberate, since one scan covers many domains, but each domain needs its own progress and score record.

Part II — Frontend

The frontend is a React/TypeScript app. It never talks to individual microservices directly — every request goes through one axios client (`api.ts`, `baseUrl` = the gateway) and lands on the API gateway, which proxies it to the right internal service.

15. Frontend ↔ backend linkup



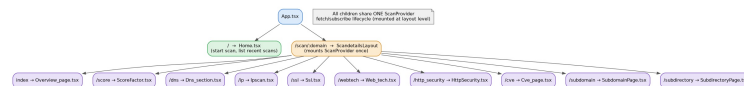
Frontend and backend linkup

The gateway is the **only** thing the browser ever talks to. It fans out to:

- `/auth/*` → Auth Service
- `/orchestrator/*` → Orchestrator Service (drives the scan lifecycle)
- `/scoring/*` → Scoring Service (reads/serves scan data to the frontend)

Because the gateway sanitizes the domain field on every proxied JSON body (strips `http(s)://` and a trailing slash), the frontend never has to worry about domain formatting before sending a request — neither does any downstream service.

16. Frontend routing



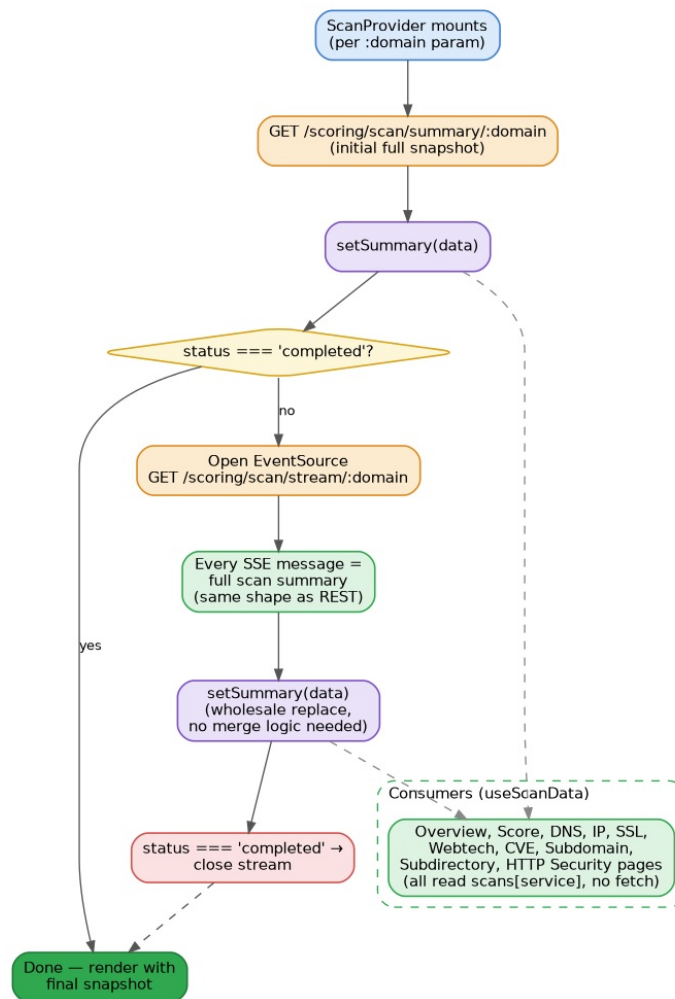
Frontend routing tree

`App.tsx` defines two branches:

- `/` → `Home.tsx` — start a new scan, list recent scans.
- `/scan/:domain` → wrapped in `ScanDetailsLayout`, which mounts a `ScanProvider` (from `ScanContext.tsx`) around every child route:
 - `/scan/:domain(index)` → `Overview_page.tsx`
 - `/scan/:domain/score` → `ScoreFactor.tsx`
 - `/scan/:domain/dns, /ip, /ssl, /webtech, /http_security, /cve, /subdomain, /subdirectory` → one page per scanner service.

Mounting `ScanProvider` once at the layout level — rather than in each page — is what lets every child route below it share one `fetch/subscribe` lifecycle instead of each page independently hitting the API.

17. Centralized data fetching — ScanContext



ScanContext data flow

This is the key architectural decision on the frontend: **only one component fetches data**. Every service page, the overview page, and the score page all just read from a shared React context (useScanContext, re-exported as useScanData from Userscandata.tsx).

ScanProvider's lifecycle, per :domain param:

1. On mount, calls GET /scoring/scan/summary/:domain once for the initial full snapshot and calls setSummary(data).
2. If the scan isn't finished (status !== 'completed'), it opens an EventSource to GET /scoring/scan/stream/:domain.
3. Every SSE message is a **full scan summary** — the exact same shape as the REST response, never a partial/raw update — so the client can just call setSummary(data) wholesale on every message, no merging logic needed.
4. The stream self-closes once the summary's status === 'completed'.

Every service page follows the same consumption pattern — none of them ever make their own network call:

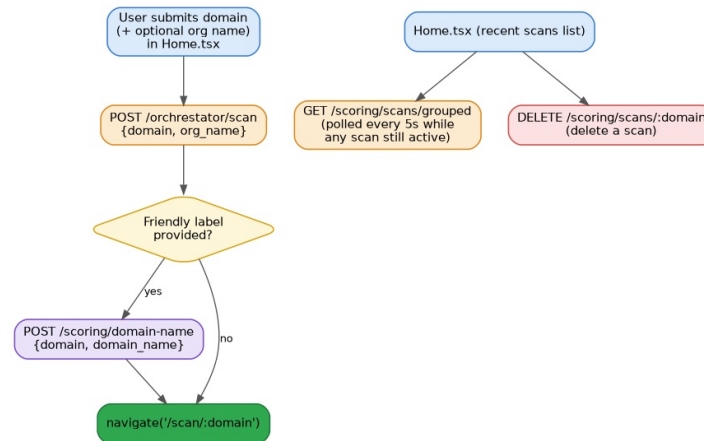
```

const { loading, scans } = useScanData();
const scan = scans['<service_name>'];
if (!scan || scan.status !== 'completed' || !scan.results) {
  return <LoadingCard service="..." status={scan?.status ||
'queued'} />;
}
// render scan.results

```

This pattern is used identically by `Dns_section.tsx`, `Ipscan.tsx`, `Ssl.tsx`, `Web_tech.tsx`, `CvePage.tsx`, `SubdirectoryPage.tsx`, `SubdomainPage.tsx`, and `HttpSecurity.tsx`.

18. Starting a scan from the UI



Start scan flow

`Home.tsx` is responsible for:

- **Starting a scan** — `POST /orchrestator/scan` with `{ domain, org_name }`, optionally followed by `POST /scoring/domain-name` to attach a friendly label, then `navigate('/scan/:domain')`.
- **Listing recent scans** — `GET /scoring/scans/grouped`, polling every 5 seconds while any listed scan is still active.
- **Deleting a scan** — `DELETE /scoring/scans/:domain`.

19. Overview page logic

`Overview_page.tsx` splits the 7 scanner services into two groups, mirroring the backend's own `FAST_SERVICES` / `SLOW_SERVICES` split in the scoring service:

- `fastServices = [dns, ip, ssl, webtech]`
- `slowServices = [subdirectory, cve, http_security]`

While the 4 fast services aren't all done yet, it shows a "Scanning..." progress screen. Once they are, it shows the score, grade, and a per-service status grid — and only renders the final `Overview_score` once **every** service (fast *and* slow) has completed.

Frontend summary

- **One context owns all fetching** (`ScanContext`) — REST for the initial snapshot, SSE for live updates. Every page below it is a pure consumer keyed off `scans[service]`, never making its own network call.
- **Subdomain pooling is fully wired end-to-end**: the orchestrator waits on subdomain discovery (with a timeout fallback), merges discovered subdomains with the root domain, and scans + scores every one of them individually — the frontend then reflects that same rollout through the identical `summary/stream` shape, regardless of whether it's describing the root domain or a subdomain.